Feature extraction

ml_dataset/scripts/feature_extraction_code.py

```python
#!/usr/bin/env python3
"""
FEATURE EXTRACTION CODE FOR THE PIPELINE

Output:
  - participant_features_v6.csv  (430 rows × ~130 cols)
  - movie_features_v6.csv        (10 rows × ~260 cols, mean + std)

"""
from __future__ import annotations
import argparse
import csv
import json
import math
import os
import statistics
import sys
from pathlib import Path
from typing import Dict, List, Optional, Tuple

SCRIPT_DIR = Path(__file__).resolve().parent
ML_DIR = SCRIPT_DIR.parent
PROJECT = ML_DIR.parent
OPENFACE_DIR = PROJECT / "OpenFace"
QUANTUM_DIR = PROJECT / "Quantum"
STUDY_DIR = PROJECT / "Study_data"
META_DIR = ML_DIR / "metadata"
OUT_DIR = ML_DIR / "data" / "model_ready" / "movie_success_v6"

CONDITIONS = [
    "AMUSEMENT", "ANGER", "AWE", "DISGUST", "ENTHUSIASM",
    "FEAR", "LIKING", "NEUTRAL", "SADNESS", "SURPRISE",
]
#  UTILITIES

def safe_mean(vals):
    return statistics.mean(vals) if vals else None

def safe_std(vals):
    return statistics.stdev(vals) if len(vals) > 1 else None

def slope(vals):
    n = len(vals)
    if n < 2: return None
    x_mean = (n - 1) / 2.0
    y_mean = sum(vals) / n
    num = sum((i - x_mean) * (v - y_mean) for i, v in enumerate(vals))
    den = sum((i - x_mean) ** 2 for i in range(n))
    return num / den if den > 0 else 0.0

def count_peaks(vals, threshold=None):
    if len(vals) < 3: return 0
    if threshold is None:
        threshold = safe_mean(vals) + (safe_std(vals) or 0)
    return sum(1 for i in range(1, len(vals) - 1)
               if vals[i] > vals[i-1] and vals[i] > vals[i+1] and vals[i] > threshold)

def first_above(vals, threshold, sample_rate):
    for i, v in enumerate(vals):
        if v > threshold:
            return i / sample_rate
    return None

def fraction_above(vals, threshold):
    return sum(1 for v in vals if v > threshold) / len(vals) if vals else 0.0

def fraction_below(vals, threshold):
    return sum(1 for v in vals if v < threshold) / len(vals) if vals else 0.0

def half_split_means(vals):
    if len(vals) < 4: return None, None
    mid = len(vals) // 2
```

```
    return safe_mean(vals[:mid]), safe_mean(vals[mid:])

def rmssd(intervals):
    if len(intervals) < 2: return None
    diffs = [(intervals[i+1] - intervals[i]) ** 2 for i in range(len(intervals) - 1)]
    return math.sqrt(sum(diffs) / len(diffs))

def pnn50(intervals):
    if len(intervals) < 2: return None
    diffs = [abs(intervals[i+1] - intervals[i]) for i in range(len(intervals) - 1)]
    return sum(1 for d in diffs if d > 0.050) / len(diffs)

def dominant_runs(labels):
    if not labels: return 0, None
    runs = 1
    lengths = [1]
    for i in range(1, len(labels)):
        if labels[i] != labels[i-1]:
            runs += 1
            lengths.append(1)
        else:
            lengths[-1] += 1
    return runs, safe_mean(lengths)

def delta(stim_val, base_val):
    """Return stim - base if both numeric, else None."""
    if stim_val is None or base_val is None:
        return None
    try:
        return float(stim_val) - float(base_val)
    except (ValueError, TypeError):
        return None

#  OPENFACE — shared core extractor for stimulus + baseline

def _openface_core(fpath: Path, duration_s_fallback: float = 60.0) -> Dict:
    """Returns raw AU/pose/gaze series from an OpenFace CSV."""
    if not fpath.exists():
        return {}
    rows = []
    with open(fpath) as f:
        reader = csv.DictReader(f)
        for row in reader:
            rows.append(row)
    if len(rows) < 30:
        return {}

    fps = 60.0
    n = len(rows)
    duration_s = n / fps

    AU_INTENSITY = ["AU01_r", "AU02_r", "AU04_r", "AU05_r", "AU06_r", "AU07_r",
                    "AU09_r", "AU10_r", "AU12_r", "AU14_r", "AU15_r", "AU17_r",
                    "AU20_r", "AU23_r", "AU25_r", "AU26_r", "AU45_r"]

    au_series = {}
    for au in AU_INTENSITY:
        col = " " + au
        vals = []
        for r in rows:
            try:
                vals.append(float(r.get(col, r.get(au, 0))))
            except (ValueError, TypeError):
                vals.append(0.0)
        au_series[au] = vals

    pose = {"pose_Rx": [], "pose_Ry": [], "pose_Rz": []}
    for r in rows:
        for pc in pose:
            col = " " + pc
            try:
                pose[pc].append(float(r.get(col, r.get(pc, 0))))
            except (ValueError, TypeError):
                pose[pc].append(0.0)

    gaze_x, gaze_y, conf = [], [], []
    for r in rows:
        try:
```

```
            gaze_x.append(float(r.get(" gaze_angle_x", r.get("gaze_angle_x", 0))))
            gaze_y.append(float(r.get(" gaze_angle_y", r.get("gaze_angle_y", 0))))
            conf.append(float(r.get(" confidence", r.get("confidence", 0))))
        except (ValueError, TypeError):
            gaze_x.append(0.0); gaze_y.append(0.0); conf.append(0.0)

    return {
        "n": n, "fps": fps, "duration_s": duration_s,
        "au": au_series, "pose": pose,
        "gaze_x": gaze_x, "gaze_y": gaze_y, "conf": conf,
    }


def _openface_level_features(core: Dict) -> Dict[str, Optional[float]]:
    """Compute just the LEVEL features (used for baseline comparison)."""
    if not core:
        return {}
    au = core["au"]
    feat = {}
    # Smile
    feat["smile_fraction"] = fraction_above(au["AU12_r"], 0.5)
    duchenne = [1 if au["AU12_r"][i] > 0.5 and au["AU06_r"][i] > 0 else 0
                for i in range(core["n"])]
    feat["duchenne_fraction"] = sum(duchenne) / core["n"]
    # AU means (for bc versions)
    for key in ["AU04_r", "AU09_r", "AU12_r", "AU15_r", "AU25_r"]:
        feat[f"{key.lower()}_mean"] = safe_mean(au[key])
    # Blink rate
    au45_active = [1 if v > 0.5 else 0 for v in au["AU45_r"]]
    blink_onsets = sum(1 for i in range(1, core["n"])
                       if au45_active[i] == 1 and au45_active[i-1] == 0)
    feat["blink_rate_per_min"] = blink_onsets / core["duration_s"] * 60
    return feat


def extract_openface_stim(pid: int, cond: str,
                          baseline_levels: Dict) -> Dict[str, Optional[float]]:
    """Stimulus-stage OpenFace features + baseline-corrected versions."""
    fpath = OPENFACE_DIR / str(pid) / f"{pid}_{cond}_STIMULUS.csv"
    core = _openface_core(fpath)
    if not core:
        return {}

    feat = {}
    prefix = "of_"
    au = core["au"]
    n = core["n"]
    fps = core["fps"]
    duration_s = core["duration_s"]

    # Smile dynamics
    au12, au06 = au["AU12_r"], au["AU06_r"]
    smile_threshold = 0.5
    smile_active = [1 if v > smile_threshold else 0 for v in au12]
    smile_onsets = sum(1 for i in range(1, n)
                       if smile_active[i] == 1 and smile_active[i-1] == 0)
    feat[prefix + "smile_rate_per_min"] = smile_onsets / duration_s * 60

    smile_fraction = fraction_above(au12, smile_threshold)
    feat[prefix + "smile_fraction"] = smile_fraction
    feat[prefix + "smile_fraction_bc"] = delta(smile_fraction, baseline_levels.get("smile_fraction"))

    # Duchenne smile: AU12 > 0.5 AND AU06 > 0
    duchenne = [1 if au12[i] > smile_threshold and au06[i] > 0 else 0 for i in range(n)]
    duchenne_frac = sum(duchenne) / n
    feat[prefix + "duchenne_fraction"] = duchenne_frac
    feat[prefix + "duchenne_fraction_bc"] = delta(duchenne_frac, baseline_levels.get("duchenne_fraction"))

    # Episode durations
    episode_lengths = []
    current_len = 0
    for v in smile_active:
        if v == 1:
            current_len += 1
        elif current_len > 0:
            episode_lengths.append(current_len / fps); current_len = 0
    if current_len > 0:
        episode_lengths.append(current_len / fps)
```

```python
    feat[prefix + "smile_mean_duration_s"] = safe_mean(episode_lengths)
    feat[prefix + "smile_longest_s"] = max(episode_lengths) if episode_lengths else 0.0
    feat[prefix + "smile_onset_latency_s"] = first_above(au12, smile_threshold, fps)
    feat[prefix + "smile_peak_timing"] = (au12.index(max(au12)) / n) if au12 else None

    first_half, second_half = half_split_means(au12)
    feat[prefix + "smile_first_half"] = first_half
    feat[prefix + "smile_second_half"] = second_half
    if first_half is not None and second_half is not None:
        feat[prefix + "smile_buildup"] = second_half - first_half

    # ── AU velocities (dynamics) ──
    for au_name in ["AU12_r", "AU04_r", "AU09_r", "AU25_r"]:
        series = au[au_name]
        velocity = [abs(series[i] - series[i-1]) for i in range(1, len(series))]
        short = au_name.lower().replace("_r", "")
        feat[prefix + f"{short}_velocity"] = safe_mean(velocity)

    # ── Individual negative AU means (DON'T sum them) ──
    #   AU04 = brow furrow (concern, concentration)
    #   AU09 = nose wrinkle (disgust)
    #   AU15 = lip corner depress (sadness)
    for au_name, label in [("AU04_r", "brow_furrow"), ("AU09_r", "nose_wrinkle"),
                           ("AU15_r", "lip_depress")]:
        series = au[au_name]
        m = safe_mean(series)
        feat[prefix + f"{label}_mean"] = m
        feat[prefix + f"{label}_max"] = max(series) if series else None
        feat[prefix + f"{label}_mean_bc"] = delta(m, baseline_levels.get(f"{au_name.lower()}_mean"))
        feat[prefix + f"{label}_onset_latency_s"] = first_above(series, 1.0, fps)

    # ── Blink features ──
    au45 = au["AU45_r"]
    blink_active = [1 if v > 0.5 else 0 for v in au45]
    blink_onsets = sum(1 for i in range(1, n)
                       if blink_active[i] == 1 and blink_active[i-1] == 0)
    blink_rate = blink_onsets / duration_s * 60
    feat[prefix + "blink_rate_per_min"] = blink_rate
    feat[prefix + "blink_rate_bc"] = delta(blink_rate, baseline_levels.get("blink_rate_per_min"))

    blink_times = [i / fps for i in range(1, n)
                   if blink_active[i] == 1 and blink_active[i-1] == 0]
    ibis = [blink_times[i] - blink_times[i-1] for i in range(1, len(blink_times))]
    feat[prefix + "blink_interval_std"] = safe_std(ibis)

    # ── Head movement ──
    pose = core["pose"]
    head_vel = []
    for i in range(1, n):
        v = sum(abs(pose[pc][i] - pose[pc][i-1]) for pc in pose)
        head_vel.append(v)
    feat[prefix + "head_velocity_mean"] = safe_mean(head_vel)
    # Fixed: use fraction_below directly, not convoluted inverse
    feat[prefix + "head_stillness_fraction"] = fraction_below(head_vel, 0.01)

    # ── Gaze stability (combined horizontal + vertical) ──
    gx, gy = core["gaze_x"], core["gaze_y"]
    gaze_mag = [math.sqrt(gx[i]**2 + gy[i]**2) for i in range(len(gx))]
    feat[prefix + "gaze_stability"] = safe_std(gaze_mag)

    # ── Confidence (quality indicator) ──
    feat[prefix + "confidence_mean"] = safe_mean(core["conf"])

    return feat


def extract_openface_baseline(pid: int) -> Dict[str, Optional[float]]:
    """Extract the level features only from the BASELINE OpenFace file."""
    fpath = OPENFACE_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS.csv"
    core = _openface_core(fpath)
    return _openface_level_features(core)

#  QUANTUM — baseline + stimulus

def _quantum_core(fpath: Path):
    if not fpath.exists():
        return {}
    with open(fpath) as f:
```

```
        data = json.load(f)
    frames = data.get("frames", [])
    if len(frames) < 30:
        return {}

    EMOTIONS = ["neutral", "anger", "disgust", "happiness", "sadness", "surprise"]
    n = len(frames)
    fps = 60.0
    duration_s = n / fps

    emo_series = {e: [] for e in EMOTIONS}
    dominant = []
    yaw, pitch = [], []
    for frame in frames:
        faces = frame.get("faces", [])
        if not faces:
            for e in EMOTIONS:
                emo_series[e].append(0.0)
            dominant.append("no_face"); continue
        face = faces[0]
        em = face.get("emotions", {})
        for e in EMOTIONS:
            emo_series[e].append(float(em.get(e, 0)))
        dominant.append(face.get("emotionName", "neutral"))
        hp = face.get("headPose", {})
        if hp:
            yaw.append(float(hp.get("yaw", 0)))
            pitch.append(float(hp.get("pitch", 0)))

    return {
        "n": n, "fps": fps, "duration_s": duration_s,
        "emo": emo_series, "dominant": dominant,
        "yaw": yaw, "pitch": pitch, "emotions": EMOTIONS,
    }


def _quantum_level_features(core):
    if not core: return {}
    return {f"{e}_mean": safe_mean(core["emo"][e]) for e in core["emotions"]}


def extract_quantum_stim(pid, cond, baseline_levels):
    fpath = QUANTUM_DIR / str(pid) / f"{pid}_{cond}_STIMULUS.json"
    core = _quantum_core(fpath)
    if not core: return {}

    feat = {}
    prefix = "q_"
    n = core["n"]; fps = core["fps"]; duration_s = core["duration_s"]
    emo = core["emo"]

    # Per-emotion stats + baseline-corrected
    for e in core["emotions"]:
        m = safe_mean(emo[e])
        feat[prefix + f"{e}_mean"] = m
        feat[prefix + f"{e}_max"] = max(emo[e]) if emo[e] else None
        feat[prefix + f"{e}_mean_bc"] = delta(m, baseline_levels.get(f"{e}_mean"))

    # Engagement dynamics
    non_neutral = [sum(emo[e][i] for e in core["emotions"] if e != "neutral")
                   for i in range(n)]
    feat[prefix + "non_neutral_fraction"] = fraction_above(non_neutral, 0.1)
    feat[prefix + "neutral_escape_time_s"] = first_above(non_neutral, 0.2, fps)

    # Entropy (with proper normalisation)
    entropies = []
    for i in range(n):
        probs = [emo[e][i] for e in core["emotions"]]
        total = sum(probs)
        if total > 0:
            probs = [p / total for p in probs]
            h = -sum(p * math.log2(p + 1e-10) for p in probs if p > 0)
            entropies.append(h)
    feat[prefix + "entropy_mean"] = safe_mean(entropies)

    # Transitions — use RATE per minute, not raw count
    transitions, mean_run_len = dominant_runs(core["dominant"])
    feat[prefix + "transition_rate_per_min"] = transitions / duration_s * 60 if duration_s > 0 else None
```

```
    feat[prefix + "mean_dominant_duration_s"] = (mean_run_len / fps) if mean_run_len else None

    # Happiness dynamics
    h = emo["happiness"]
    feat[prefix + "happiness_onset_s"] = first_above(h, 0.1, fps)
    feat[prefix + "happiness_slope"] = slope(h)
    h_first, h_second = half_split_means(h)
    feat[prefix + "happiness_buildup"] = (h_second - h_first) if h_first is not None and h_second is not None else None

    # Peak timing
    total_emo = [sum(emo[e][i] for e in core["emotions"] if e != "neutral") for i in range(n)]
    if total_emo:
        feat[prefix + "peak_emotion_timing"] = total_emo.index(max(total_emo)) / n

    # Head pose stability
    feat[prefix + "head_yaw_std"] = safe_std(core["yaw"])
    feat[prefix + "head_pitch_std"] = safe_std(core["pitch"])

    return feat


def extract_quantum_baseline(pid):
    fpath = QUANTUM_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS.json"
    core = _quantum_core(fpath)
    return _quantum_level_features(core)
#  EMPATICA — baseline + stimulus

def _empatica_core(fpath):
    if not fpath.exists():
        return {}
    with open(fpath) as f:
        data = json.load(f)

    eda_vals = [float(x[1]) for x in data.get("EDA", []) if len(x) >= 2]
    ibi_vals = [float(x[1]) for x in data.get("IBI", [])
                if len(x) >= 2 and 0.3 < float(x[1]) < 2.0]
    temp_vals = [float(x[1]) for x in data.get("TEMP", []) if len(x) >= 2]
    bvp_vals = [float(x[1]) for x in data.get("BVP", []) if len(x) >= 2]
    acc_raw = data.get("ACC", [])
    acc_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
               for x in acc_raw if len(x) >= 4]

    return {"eda": eda_vals, "ibi": ibi_vals, "temp": temp_vals,
            "bvp": bvp_vals, "acc_mag": acc_mag}


def _empatica_level_features(core):
    if not core: return {}
    feat = {}
    feat["eda_mean"] = safe_mean(core["eda"]) if len(core["eda"]) > 10 else None

    # SCR rate per minute
    if len(core["eda"]) > 10:
        scr_count = count_peaks(core["eda"])
        feat["scr_rate_per_min"] = scr_count / (len(core["eda"]) / 4.0) * 60
    else:
        feat["scr_rate_per_min"] = None

    if len(core["ibi"]) > 5:
        feat["hr_mean"] = 60.0 / safe_mean(core["ibi"])
        feat["rmssd"] = rmssd(core["ibi"])
        feat["pnn50"] = pnn50(core["ibi"])
    else:
        feat["hr_mean"] = feat["rmssd"] = feat["pnn50"] = None

    feat["temp_mean"] = safe_mean(core["temp"]) if len(core["temp"]) > 10 else None
    feat["movement_mean"] = safe_mean(core["acc_mag"]) if len(core["acc_mag"]) > 10 else None
    return feat


def extract_empatica_stim(pid, cond, baseline_levels):
    fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_EMPATICA.json"
    core = _empatica_core(fpath)
    if not core: return {}
    feat = {}
    prefix = "emp_"
    eda = core["eda"]; ibi = core["ibi"]; temp = core["temp"]
```

```
    bvp = core["bvp"]; acc_mag = core["acc_mag"]

    # EDA
    if len(eda) > 10:
        eda_mean = safe_mean(eda)
        feat[prefix + "eda_mean"] = eda_mean
        feat[prefix + "eda_mean_bc"] = delta(eda_mean, baseline_levels.get("eda_mean"))
        feat[prefix + "eda_std"] = safe_std(eda)
        feat[prefix + "eda_slope"] = slope(eda)

        scr_count = count_peaks(eda)
        scr_rate = scr_count / (len(eda) / 4.0) * 60
        feat[prefix + "scr_rate_per_min"] = scr_rate
        feat[prefix + "scr_rate_bc"] = delta(scr_rate, baseline_levels.get("scr_rate_per_min"))

        threshold = safe_mean(eda) + (safe_std(eda) or 0)
        feat[prefix + "scr_onset_latency_s"] = first_above(eda, threshold, 4.0)

        first, second = half_split_means(eda)
        feat[prefix + "eda_buildup"] = (second - first) if first is not None and second is not None else
None

    # HRV
    if len(ibi) > 5:
        hr = 60.0 / safe_mean(ibi)
        feat[prefix + "hr_mean"] = hr
        feat[prefix + "hr_mean_bc"] = delta(hr, baseline_levels.get("hr_mean"))
        feat[prefix + "ibi_mean"] = safe_mean(ibi)
        feat[prefix + "ibi_std"] = safe_std(ibi)

        r = rmssd(ibi)
        feat[prefix + "rmssd"] = r
        feat[prefix + "rmssd_bc"] = delta(r, baseline_levels.get("rmssd"))

        p = pnn50(ibi)
        feat[prefix + "pnn50"] = p
        feat[prefix + "pnn50_bc"] = delta(p, baseline_levels.get("pnn50"))

        hr_vals = [60.0 / v for v in ibi if v > 0]
        hr_first, hr_second = half_split_means(hr_vals)
        feat[prefix + "hr_reactivity"] = (hr_second - hr_first) if hr_first is not None and hr_second is
not None else None
        feat[prefix + "hr_range"] = max(hr_vals) - min(hr_vals) if hr_vals else None

    # BVP
    feat[prefix + "bvp_std"] = safe_std(bvp) if len(bvp) > 10 else None

    # Temperature — absolute level, baseline-corrected (NOT slope)
    if len(temp) > 10:
        t = safe_mean(temp)
        feat[prefix + "temp_mean"] = t
        feat[prefix + "temp_mean_bc"] = delta(t, baseline_levels.get("temp_mean"))

    # Movement / fidgeting
    if len(acc_mag) > 10:
        m = safe_mean(acc_mag)
        feat[prefix + "movement_mean"] = m
        feat[prefix + "movement_mean_bc"] = delta(m, baseline_levels.get("movement_mean"))
        feat[prefix + "movement_std"] = safe_std(acc_mag)
        # Fidget rate per minute
        fc = count_peaks(acc_mag)
        feat[prefix + "fidget_rate_per_min"] = fc / (len(acc_mag) / 32.0) * 60

    return feat


def extract_empatica_baseline(pid):
    fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_EMPATICA.json"
    return _empatica_level_features(_empatica_core(fpath))


#
#  MUSE EEG — baseline + stimulus
#

def _muse_core(fpath):
    if not fpath.exists(): return {}
    with open(fpath) as f:
```

```
        data = json.load(f)

    required = ["Alpha_AF7", "Alpha_AF8", "Beta_AF7", "Beta_AF8",
                "Theta_AF7", "Theta_AF8",
                "Alpha_TP9", "Alpha_TP10", "HeadBandOn"]
    for r in required:
        if r not in data or not data[r]:
            return {}

    n = len(data["Alpha_AF7"])
    if n < 100: return {}

    hb = data.get("HeadBandOn", [1] * n)

    def get_valid(key):
        vals = data.get(key, [])
        return [float(vals[i]) for i in range(min(len(vals), n))
                if i < len(hb) and float(hb[i]) > 0.5]

    return {
        "alpha_af7": get_valid("Alpha_AF7"), "alpha_af8": get_valid("Alpha_AF8"),
        "alpha_tp9": get_valid("Alpha_TP9"), "alpha_tp10": get_valid("Alpha_TP10"),
        "beta_af7": get_valid("Beta_AF7"), "beta_af8": get_valid("Beta_AF8"),
        "theta_af7": get_valid("Theta_AF7"), "theta_af8": get_valid("Theta_AF8"),
    }


def _muse_level_features(core):
    if not core or len(core["alpha_af7"]) < 50: return {}
    feat = {}
    # Alpha asymmetry: ln(AF8) - ln(AF7)
    alpha_asym = []
    for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"]))):
        if core["alpha_af7"][i] > 0 and core["alpha_af8"][i] > 0:
            alpha_asym.append(math.log(core["alpha_af8"][i]) - math.log(core["alpha_af7"][i]))
    feat["alpha_asym_mean"] = safe_mean(alpha_asym)

    beta_asym = []
    for i in range(min(len(core["beta_af7"]), len(core["beta_af8"]))):
        if core["beta_af7"][i] > 0 and core["beta_af8"][i] > 0:
            beta_asym.append(math.log(core["beta_af8"][i]) - math.log(core["beta_af7"][i]))
    feat["beta_asym_mean"] = safe_mean(beta_asym)

    # Frontal alpha (target of suppression)
    frontal_alpha = [(core["alpha_af7"][i] + core["alpha_af8"][i]) / 2
                     for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"])))]
    feat["frontal_alpha_mean"] = safe_mean(frontal_alpha)

    # Frontal theta
    frontal_theta = [(core["theta_af7"][i] + core["theta_af8"][i]) / 2
                     for i in range(min(len(core["theta_af7"]), len(core["theta_af8"])))]
    feat["frontal_theta_mean"] = safe_mean(frontal_theta)

    # Per-channel alpha means
    feat["alpha_af7_mean"] = safe_mean(core["alpha_af7"])
    feat["alpha_af8_mean"] = safe_mean(core["alpha_af8"])
    feat["alpha_tp9_mean"] = safe_mean(core["alpha_tp9"])
    feat["alpha_tp10_mean"] = safe_mean(core["alpha_tp10"])

    return feat


def extract_muse_stim(pid, cond, baseline_levels):
    fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_MUSE.json"
    core = _muse_core(fpath)
    if not core or len(core["alpha_af7"]) < 50: return {}

    feat = {}
    prefix = "eeg_"

    # Alpha asymmetry (absolute + BASELINE-CORRECTED)
    alpha_asym = []
    for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"]))):
        if core["alpha_af7"][i] > 0 and core["alpha_af8"][i] > 0:
            alpha_asym.append(math.log(core["alpha_af8"][i]) - math.log(core["alpha_af7"][i]))

    asym_mean = safe_mean(alpha_asym)
    feat[prefix + "alpha_asym_mean"] = asym_mean
```

```
    feat[prefix + "alpha_asym_change"] = delta(asym_mean, baseline_levels.get("alpha_asym_mean"))
    feat[prefix + "alpha_asym_std"] = safe_std(alpha_asym)
    feat[prefix + "alpha_asym_slope"] = slope(alpha_asym) if len(alpha_asym) > 10 else None

    first, second = half_split_means(alpha_asym)
    feat[prefix + "alpha_asym_buildup"] = (second - first) if first is not None and second is not None
else None

    # Beta asymmetry (+ bc)
    beta_asym = []
    for i in range(min(len(core["beta_af7"]), len(core["beta_af8"]))):
        if core["beta_af7"][i] > 0 and core["beta_af8"][i] > 0:
            beta_asym.append(math.log(core["beta_af8"][i]) - math.log(core["beta_af7"][i]))
    beta_mean = safe_mean(beta_asym)
    feat[prefix + "beta_asym_mean"] = beta_mean
    feat[prefix + "beta_asym_change"] = delta(beta_mean, baseline_levels.get("beta_asym_mean"))

    # Frontal alpha SUPPRESSION = baseline - stim (positive = more alertness during stim)
    frontal_alpha_stim = [(core["alpha_af7"][i] + core["alpha_af8"][i]) / 2
                          for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"])))]
    fa_mean = safe_mean(frontal_alpha_stim)
    feat[prefix + "frontal_alpha_mean"] = fa_mean
    base_fa = baseline_levels.get("frontal_alpha_mean")
    if fa_mean is not None and base_fa is not None:
        feat[prefix + "frontal_alpha_suppression"] = base_fa - fa_mean

    # Frontal theta change (positive = more emotional processing)
    frontal_theta_stim = [(core["theta_af7"][i] + core["theta_af8"][i]) / 2
                          for i in range(min(len(core["theta_af7"]), len(core["theta_af8"])))]
    ft_mean = safe_mean(frontal_theta_stim)
    feat[prefix + "frontal_theta_mean"] = ft_mean
    feat[prefix + "frontal_theta_change"] = delta(ft_mean, baseline_levels.get("frontal_theta_mean"))

    # Per-channel alpha (baseline-corrected versions)
    for ch in ["af7", "af8", "tp9", "tp10"]:
        m = safe_mean(core[f"alpha_{ch}"])
        feat[prefix + f"alpha_{ch}_mean"] = m
        feat[prefix + f"alpha_{ch}_mean_bc"] = delta(m, baseline_levels.get(f"alpha_{ch}_mean"))

    # Alpha variability (time-domain, no baseline needed)
    feat[prefix + "alpha_variability"] = safe_std(frontal_alpha_stim)

    return feat


def extract_muse_baseline(pid):
    fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_MUSE.json"
    return _muse_level_features(_muse_core(fpath))

#  SAMSUNG WATCH — baseline + stimulus

def _samsung_core(fpath):
    if not fpath.exists(): return {}
    with open(fpath) as f:
        data = json.load(f)
    hr = [float(x[1]) for x in data.get("heartRate", [])
          if len(x) >= 2 and 30 < float(x[1]) < 200]
    ppi = [float(x[1]) / 1000.0 for x in data.get("PPInterval", [])
           if len(x) >= 2 and 300 < float(x[1]) < 2000]
    acc_raw = data.get("acc", [])
    acc_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
               for x in acc_raw if len(x) >= 4]
    gyr_raw = data.get("gyr", [])
    gyr_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
               for x in gyr_raw if len(x) >= 4]
    return {"hr": hr, "ppi": ppi, "acc_mag": acc_mag, "gyr_mag": gyr_mag}


def _samsung_level_features(core):
    if not core: return {}
    feat = {}
    feat["hr_mean"] = safe_mean(core["hr"]) if len(core["hr"]) > 10 else None
    feat["ppi_mean"] = safe_mean(core["ppi"]) if len(core["ppi"]) > 10 else None
    feat["movement_mean"] = safe_mean(core["acc_mag"]) if len(core["acc_mag"]) > 30 else None
    return feat


def extract_samsung_stim(pid, cond, baseline_levels):
```

```
    fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_SAMSUNG_WATCH.json"
    core = _samsung_core(fpath)
    if not core: return {}

    feat = {}
    prefix = "sw_"

    hr = core["hr"]
    if len(hr) > 10:
        m = safe_mean(hr)
        feat[prefix + "hr_mean"] = m
        feat[prefix + "hr_mean_bc"] = delta(m, baseline_levels.get("hr_mean"))
        feat[prefix + "hr_std"] = safe_std(hr)
        feat[prefix + "hr_range"] = max(hr) - min(hr)
        first, second = half_split_means(hr)
        feat[prefix + "hr_reactivity"] = (second - first) if first is not None and second is not None else
None
        feat[prefix + "hr_peak_timing"] = hr.index(max(hr)) / len(hr)

    ppi = core["ppi"]
    if len(ppi) > 10:
        m = safe_mean(ppi)
        feat[prefix + "ppi_mean"] = m
        feat[prefix + "ppi_mean_bc"] = delta(m, baseline_levels.get("ppi_mean"))
        feat[prefix + "ppi_std"] = safe_std(ppi)
        feat[prefix + "ppi_rmssd"] = rmssd(ppi)

    acc_mag = core["acc_mag"]
    if len(acc_mag) > 30:
        m = safe_mean(acc_mag)
        feat[prefix + "movement_mean"] = m
        feat[prefix + "movement_mean_bc"] = delta(m, baseline_levels.get("movement_mean"))
        feat[prefix + "movement_std"] = safe_std(acc_mag)
        # Burst rate per minute
        bc = count_peaks(acc_mag)
        feat[prefix + "movement_burst_rate_per_min"] = bc / (len(acc_mag) / 33.4) * 60

    if len(core["gyr_mag"]) > 30:
        feat[prefix + "gyro_variability"] = safe_std(core["gyr_mag"])

    return feat


def extract_samsung_baseline(pid):
    fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_SAMSUNG_WATCH.json"
    return _samsung_level_features(_samsung_core(fpath))

#  QUESTIONNAIRE (demographics, self-report, context)

def load_questionnaire_data():
    meta_dir = PROJECT / "ml_dataset" / "metadata"
    demo = {}
    with open(meta_dir / "participants.csv") as f:
        for row in csv.DictReader(f):
            pid = int(row["participant_id"])
            demo[pid] = row

    NEVER = 24 * 60  # 1440 min: encode "never today" for missing physiological state

    def parse_mins(s, default_if_missing=None):
        if not s or not s.strip():
            return default_if_missing
        parts = s.strip().split(":")
        try:
            return int(parts[0]) * 60 + int(parts[1])
        except (ValueError, IndexError):
            return default_if_missing

    quest_data = {}
    for pid_dir in sorted(STUDY_DIR.iterdir()):
        if not pid_dir.is_dir() or not pid_dir.name.isdigit():
            continue
        pid = int(pid_dir.name)
        qpath = pid_dir / f"{pid}_QUESTIONNAIRES.json"
        if not qpath.exists():
            continue
        with open(qpath) as f:
            qdata = json.load(f)
```

```
        meta = qdata.get("metadata", {})
        movie_order = meta.get("movie_order", [])
        familiarity = meta.get("movies_seen_before_study", {})
        d = demo.get(pid, {})

        baseline_emo = {}
        for entry in qdata.get("questionnaires", []):
            if entry["movie"] == "BASELINE":
                baseline_emo = entry.get("emotions", {})
                break

        # Parse physiological state ONCE per participant (same for all conditions)
        # MISSING caffeine/cigarette/activity → 1440 (never today)
        participant_state = {
            "participant_age": int(d.get("age", 0)) if d.get("age") else None,
            "participant_gender": 1 if d.get("gender") == "male" else 0,
            "wearing_glasses": 1 if meta.get("wearing_glasses") else 0,
            "minutes_from_wakeup": parse_mins(meta.get("time_from_wake_up", "")),
            # Missing = "never today" → encode as NEVER (not imputed median)
            "minutes_from_caffeine": parse_mins(meta.get("time_from_caffeine", ""),
default_if_missing=NEVER),
            "minutes_from_cigarette": parse_mins(meta.get("time_from_cigarette", ""),
default_if_missing=NEVER),
            "minutes_from_activity": parse_mins(meta.get("time_from_activity", ""),
default_if_missing=NEVER),
            "minutes_from_meal": parse_mins(meta.get("time_from_meal", "")),
        }

        for entry in qdata.get("questionnaires", []):
            cond = entry["movie"]
            if cond == "BASELINE":
                continue
            emotions = entry.get("emotions", {})
            sam = entry.get("sam", {})
            feat = dict(participant_state)
            # Self-report emotions
            for emo, val in emotions.items():
                feat[f"sr_emo_{emo.lower()}"] = val
                feat[f"sr_emo_bc_{emo.lower()}"] = val - baseline_emo.get(emo, 0)
            feat["sr_sam_valence"] = sam.get("VALENCE")
            feat["sr_sam_arousal"] = sam.get("AROUSAL")
            feat["sr_sam_motivation"] = sam.get("MOTIVATION")
            feat["viewing_order"] = movie_order.index(cond) if cond in movie_order else None
            feat["movie_familiarity"] = familiarity.get(cond, 0)
            quest_data[(pid, cond)] = feat

    return quest_data

#  MAIN

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--pid", type=int, help="Single participant ID (debug)")
    parser.add_argument("--condition", type=str, help="Single condition (debug)")
    args = parser.parse_args()

    # Condition → movie mapping
    cond_to_movie = {}
    with open(META_DIR / "scenes.csv") as f:
        for row in csv.DictReader(f):
            if row["scene_id"] != "S000":
                cond_to_movie[row["condition"]] = row["movie_id"]

    # Targets
    targets = {}
    with open(META_DIR / "movie_financials_placeholder.csv") as f:
        for row in csv.DictReader(f):
            if row["movie_id"] and row["movie_id"] != "M000":
                targets[row["movie_id"]] = {
                    "budget_usd": row.get("budget_usd", ""),
                    "revenue_usd": row.get("revenue_usd", ""),
                    "roi_percent": row.get("roi_percent", ""),
                    "success_class": row.get("success_class", ""),
                }

    pids = sorted([int(d) for d in os.listdir(str(STUDY_DIR)) if d.isdigit()])
    if args.pid:
        pids = [args.pid]
```

```
    conditions = [args.condition.upper()] if args.condition else CONDITIONS

    print(f"V6 extraction: {len(pids)} participants × {len(conditions)} conditions", flush=True)
    print("Loading questionnaire data...", flush=True)
    quest_lookup = load_questionnaire_data()

    all_records = []
    for pi, pid in enumerate(pids):
        # Extract baseline features ONCE per participant
        bl_of = extract_openface_baseline(pid)
        bl_q  = extract_quantum_baseline(pid)
        bl_emp = extract_empatica_baseline(pid)
        bl_muse = extract_muse_baseline(pid)
        bl_sw = extract_samsung_baseline(pid)

        for cond in conditions:
            mid = cond_to_movie.get(cond, "")
            if not mid or mid not in targets:
                continue

            of_feat = extract_openface_stim(pid, cond, bl_of)
            q_feat  = extract_quantum_stim(pid, cond, bl_q)
            emp_feat = extract_empatica_stim(pid, cond, bl_emp)
            muse_feat = extract_muse_stim(pid, cond, bl_muse)
            sw_feat = extract_samsung_stim(pid, cond, bl_sw)
            quest = quest_lookup.get((pid, cond), {})

            # Require at least ONE modality to exist for this row
            if not any([of_feat, q_feat, emp_feat, muse_feat, sw_feat]):
                continue

            record = {
                "participant_id": pid,
                "condition": cond,
                "movie_id": mid,
                # Modality availability masks
                "has_openface": 1 if of_feat else 0,
                "has_quantum":  1 if q_feat else 0,
                "has_empatica": 1 if emp_feat else 0,
                "has_muse":     1 if muse_feat else 0,
                "has_samsung":  1 if sw_feat else 0,
            }
            record.update(of_feat)
            record.update(q_feat)
            record.update(emp_feat)
            record.update(muse_feat)
            record.update(sw_feat)
            record.update(quest)
            all_records.append(record)

        if (pi + 1) % 5 == 0 or pi == len(pids) - 1:
            print(f"  [{pi+1}/{len(pids)}] processed · {len(all_records)} records", flush=True)

    if not all_records:
        print("ERROR: no records", flush=True)
        sys.exit(1)

    # Collect all feature columns
    id_cols = {"participant_id", "condition", "movie_id"}
    feature_cols = sorted(set(k for r in all_records for k in r if k not in id_cols))

    print(f"\nParticipant-level: {len(all_records)} rows × {len(feature_cols)} features", flush=True)

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    out_cols = ["participant_id", "condition", "movie_id"] + feature_cols
    p_path = OUT_DIR / "participant_features_v6.csv"
    with open(p_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=out_cols, extrasaction="ignore")
        writer.writeheader()
        for r in all_records:
            writer.writerow({k: (round(v, 6) if isinstance(v, float) else v)
                             for k, v in r.items()})
    print(f"Wrote {p_path}", flush=True)

    # Aggregate to movie level
    from collections import defaultdict
    by_movie = defaultdict(list)
    for r in all_records:
```

```
        by_movie[r["movie_id"]].append(r)

    numeric_feat_cols = []
    for col in feature_cols:
        vals = [r.get(col) for r in all_records if r.get(col) is not None and r.get(col) != ""]
        if vals:
            try:
                float(vals[0])
                numeric_feat_cols.append(col)
            except (ValueError, TypeError):
                pass

    movie_records = []
    for mid in sorted(targets.keys()):
        records = by_movie.get(mid, [])
        if not records:
            continue
        row = {"movie_id": mid, "condition": records[0]["condition"],
               "n_participants": len(records)}
        row.update(targets[mid])
        for col in numeric_feat_cols:
            vals = [float(r[col]) for r in records
                    if r.get(col) is not None and r[col] != ""]
            if vals:
                row[f"{col}__mean"] = round(statistics.mean(vals), 6)
                row[f"{col}__std"] = round(statistics.stdev(vals), 6) if len(vals) > 1 else 0.0
            else:
                row[f"{col}__mean"] = None
                row[f"{col}__std"] = None
        movie_records.append(row)

    if movie_records:
        m_cols = list(movie_records[0].keys())
        m_path = OUT_DIR / "movie_features_v6.csv"
        with open(m_path, "w", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=m_cols)
            writer.writeheader()
            for r in movie_records:
                writer.writerow({k: (round(v, 6) if isinstance(v, float) else v)
                                 for k, v in r.items()})
        agg_count = sum(1 for c in m_cols if c.endswith("__mean") or c.endswith("__std"))
        print(f"Wrote {m_path}: {len(movie_records)} movies × {agg_count} aggregated features",
flush=True)

    # Summary
    import re
    groups = {
        "OpenFace (of_)": "^of_",
        "Quantum (q_)":   "^q_",
        "Empatica (emp_)":"^emp_",
        "EEG (eeg_)":     "^eeg_",
        "Samsung (sw_)":  "^sw_",
        "Self-Report (sr_)": "^sr_",
        "Participant":   "^(participant_|wearing_|minutes_|viewing_)",
        "Missing masks": "^has_",
    }
    print(f"\n{'='*60}", flush=True)
    print(f"  V6 Feature Extraction Complete", flush=True)
    print(f"{'='*60}", flush=True)
    print(f"  Participant-level: {len(all_records)} × {len(feature_cols)}", flush=True)
    print(f"  Movie-level: {len(movie_records)} × {agg_count}", flush=True)
    print(f"\n  Feature breakdown:", flush=True)
    for label, pat in groups.items():
        n = sum(1 for c in feature_cols if re.match(pat, c))
        print(f"    {label:25s}  {n:3d}", flush=True)


if __name__ == "__main__":
    main()
```

Simulate 40 movies (ML, n = 50 corpus)

ml_dataset/scripts/simulate_40_movies_ML.py

```python
#!/usr/bin/env python3
"""
SIMULATION OF 40 SYNTHETIC MOVIES FOR ML TRAINING

Approach: Two-level hierarchical class-conditional sampling using
gaussian copulas. The copula approach preserves BOTH:
  - Each feature's marginal distribution (by mapping through the empirical CDF)
  - The correlation structure between features (by sampling correlated normals)

Outputs 3 file
  - scene_movie_metadata_v6_synthetic.csv   (40 rows)
  - participant_features_v6_synthetic.csv   (40 × 43 = 1720 rows)
  - movie_features_v6_synthetic.csv         (40 rows)

All synthetic rows include is_synthetic=1.
"""
from __future__ import annotations
import csv
import math
import random
import statistics
from pathlib import Path
from collections import Counter, defaultdict
import warnings

import numpy as np

warnings.filterwarnings("ignore")

# ──────────────────────────────────────────────────────────────────────
PROJECT = Path(__file__).resolve().parent.parent.parent
DATA_DIR = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v6"

RANDOM_SEED = 42
N_SYNTHETIC = 40
N_PARTICIPANTS = 43
SYNTHETIC_PARTICIPANT_ID_START = 1000
TIER_THRESHOLD = 7.5
CPI_BASE = 322.0
CPI = {1978: 65.2, 1979: 72.6, 1983: 99.6, 1988: 118.3, 1993: 144.5,
       1996: 156.9, 1998: 163.0, 1999: 166.6, 2006: 201.6, 2012: 229.6,
       2025: 322.0}

random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

def phi(z):
    """Standard normal CDF (vectorised over numpy array)."""
    z = np.asarray(z, dtype=float)
    return 0.5 * (1.0 + np.vectorize(math.erf)(z / math.sqrt(2.0)))


def phi_inv(p):
    """
    Inverse standard normal CDF using Beasley-Springer-Moro approximation.
    Vectorised over numpy array. Accuracy ~1e-9 in mid-range.
    """
    p = np.clip(np.asarray(p, dtype=float), 1e-12, 1 - 1e-12)

    a = [-39.6968302866538, 220.946098424521, -275.928510446969,
         138.357751867269, -30.6647980661472, 2.50662827745924]
    b = [-54.4760987982241, 161.585836858041, -155.698979859887,
         66.8013118877197, -13.2806815528857]
    c = [-0.00778489400243029, -0.322396458041136, -2.40075827716184,
         -2.54973253934373, 4.37466414146497, 2.93816398269878]
    d = [0.00778469570904146, 0.32246712907004, 2.445134137143, 3.75440866190742]

    p_low = 0.02425
    p_high = 1 - p_low
    out = np.empty_like(p)

    # Mid region
    mid = (p >= p_low) & (p <= p_high)
```

```
    if np.any(mid):
        q = p[mid] - 0.5
        r = q * q
        num = ((((a[0]*r + a[1])*r + a[2])*r + a[3])*r + a[4])*r + a[5]
        den = ((((b[0]*r + b[1])*r + b[2])*r + b[3])*r + b[4])*r + 1
        out[mid] = num * q / den

    # Lower tail
    low = p < p_low
    if np.any(low):
        q = np.sqrt(-2 * np.log(p[low]))
        num = ((((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
        den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
        out[low] = num / den

    # Upper tail (mirror of lower)
    high = p > p_high
    if np.any(high):
        q = np.sqrt(-2 * np.log(1 - p[high]))
        num = ((((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
        den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
        out[high] = -num / den

    return out


# ──────────────────────────────────────────────────────────────
# COPULA UTILITIES
# ──────────────────────────────────────────────────────────────

def empirical_ranks(values):
    """
    Return rank-based percentiles in (0, 1), avoiding boundaries.
    Uses (rank + 0.5) / n convention; tied values get the same rank.
    """
    n = len(values)
    if n == 0:
        return np.array([])
    # argsort-of-argsort gives 0-indexed rank
    ranks = np.argsort(np.argsort(values)).astype(float)
    return (ranks + 0.5) / n


def inverse_empirical(percentiles, sorted_values):
    """Linear-interpolating inverse ECDF: percentile [0,1] → value in original units."""
    n = len(sorted_values)
    if n == 0:
        return np.zeros_like(percentiles)
    if n == 1:
        return np.full_like(percentiles, sorted_values[0])
    idx = percentiles * (n - 1)
    idx = np.clip(idx, 0, n - 1)
    lo = np.floor(idx).astype(int)
    hi = np.minimum(lo + 1, n - 1)
    frac = idx - lo
    return sorted_values[lo] * (1.0 - frac) + sorted_values[hi] * frac


def fit_copula(X, regularize=0.05):
    """
    Fit a Gaussian copula on real data X (n × p).
    Returns model with: cholesky factor, sorted real values per feature,
                        and a flag indicating constant features.
    """
    n, p = X.shape

    sorted_per_feat = []
    constant_value = []                  # for features with zero variance
    is_constant = np.zeros(p, dtype=bool)

    Z = np.zeros((n, p))
    for j in range(p):
        col = X[:, j]
        valid_mask = ~np.isnan(col)
        valid = col[valid_mask]

        if len(valid) == 0:
            sorted_per_feat.append(np.array([0.0]))
```

```
            constant_value.append(0.0)
            is_constant[j] = True
            continue

        sorted_col = np.sort(valid)
        sorted_per_feat.append(sorted_col)

        # Constant feature
        if sorted_col[0] == sorted_col[-1]:
            is_constant[j] = True
            constant_value.append(sorted_col[0])
            Z[:, j] = 0.0
            continue
        constant_value.append(0.0)

        # Compute z-scores via empirical CDF → Φ⁻¹
        valid_ranks = empirical_ranks(valid)
        Z_valid = phi_inv(valid_ranks)

        # Place Z_valid back into the full column; impute NaN as 0 (median in z-space)
        z_col = np.zeros(n)
        z_col[valid_mask] = Z_valid
        z_col[~valid_mask] = 0.0
        Z[:, j] = z_col

    # Correlation matrix of Z
    # Skip constant columns when computing correlation
    var = Z.var(axis=0)
    nonconst = var > 1e-10
    Z_nc = Z[:, nonconst]
    C_nc = np.corrcoef(Z_nc, rowvar=False) if Z_nc.shape[1] > 1 else np.eye(1)
    C_nc = np.nan_to_num(C_nc, nan=0.0)

    # Reconstruct full correlation matrix: identity for constant columns
    C = np.eye(p)
    nc_indices = np.where(nonconst)[0]
    for i_idx, i in enumerate(nc_indices):
        for j_idx, j in enumerate(nc_indices):
            C[i, j] = C_nc[i_idx, j_idx]

    # Regularise: shrink toward identity for numerical stability
    C = (1 - regularize) * C + regularize * np.eye(p)

    # Cholesky decomposition (with extra regularisation if needed)
    for extra in [0.0, 0.05, 0.2, 0.5]:
        try:
            C_try = (1 - extra) * C + extra * np.eye(p)
            L = np.linalg.cholesky(C_try)
            break
        except np.linalg.LinAlgError:
            continue
    else:
        raise RuntimeError("Cholesky failed even with heavy regularisation.")

    return {
        "L": L, "p": p, "n": n,
        "sorted_per_feat": sorted_per_feat,
        "is_constant": is_constant,
        "constant_value": constant_value,
    }


def sample_copula(model, n_samples):
    """Draw n_samples × p synthetic feature vectors from the fitted copula."""
    L = model["L"]
    p = model["p"]
    sorted_per_feat = model["sorted_per_feat"]
    is_constant = model["is_constant"]
    constant_value = model["constant_value"]

    # Step 1: correlated standard normals
    noise = np.random.standard_normal((n_samples, p))
    Z_new = noise @ L.T

    # Step 2: → uniform via Φ
    U_new = phi(Z_new)

    # Step 3: → original space via inverse ECDF
```

```
    X_new = np.empty((n_samples, p))
    for j in range(p):
        if is_constant[j]:
            X_new[:, j] = constant_value[j]
        else:
            X_new[:, j] = inverse_empirical(U_new[:, j], sorted_per_feat[j])

    return X_new



def load_csv(path):
    with open(path) as f:
        reader = csv.DictReader(f)
        return reader.fieldnames, list(reader)

def to_float(s):
    if s is None or s == "" or s == "None":
        return float("nan")
    try:
        return float(s)
    except ValueError:
        return float("nan")

#LOAD REAL DATA

meta_cols, meta_rows = load_csv(DATA_DIR / "scene_movie_metadata_v6.csv")
print(f"  Metadata: {len(meta_rows)} movies × {len(meta_cols)} cols")

ppt_cols, ppt_rows = load_csv(DATA_DIR / "participant_features_v6.csv")
print(f"  Participants: {len(ppt_rows)} rows × {len(ppt_cols)} cols")

movie_imdb = {r["movie_id"]: float(r["imdb_rating"]) for r in meta_rows
              if r.get("imdb_rating")}
tier_of_movie = {mid: ("high" if rating >= TIER_THRESHOLD else "lower")
                 for mid, rating in movie_imdb.items()}

movies_in_tier = defaultdict(list)
for mid, t in tier_of_movie.items():
    movies_in_tier[t].append(mid)

print("\nTier assignment:")
for t in ["high", "lower"]:
    mids = movies_in_tier[t]
    ratings = [movie_imdb[m] for m in mids]
    print(f"  {t:6s}: {len(mids)} movies, IMDb {min(ratings):.1f}–{max(ratings):.1f}")



#PREPARE PARTICIPANT-LEVEL FEATURES PER TIER

ID_COLS = {"participant_id", "condition", "movie_id"}
HAS_COLS = {c for c in ppt_cols if c.startswith("has_")}
SR_INTEGER_COLS = {c for c in ppt_cols
                   if c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_")}
SR_INTEGER_COLS.update({"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"})

feat_cols = [c for c in ppt_cols if c not in ID_COLS]
n_feat = len(feat_cols)

tier_data = defaultdict(list)
for r in ppt_rows:
    mid = r["movie_id"]
    if mid not in tier_of_movie:
        continue
    vec = np.array([to_float(r[c]) for c in feat_cols])
    tier_data[tier_of_movie[mid]].append(vec)

print(f"\nFeature matrix sizes:")
for t, vecs in tier_data.items():
    arr = np.vstack(vecs)
    print(f"  Tier '{t}': {arr.shape[0]} × {arr.shape[1]}, NaN rate {np.isnan(arr).mean():.1%}")



#FIT GAUSSIAN COPULAS


```

```
print("\nFitting Gaussian copulas (preserves marginals + correlations)...")
tier_models = {}
for t, vecs in tier_data.items():
    X = np.vstack(vecs)
    model = fit_copula(X, regularize=0.05)
    tier_models[t] = model
    n_const = int(model["is_constant"].sum())
    print(f"  Tier '{t}': fitted on {model['n']} rows × {model['p']} features  "
          f"({n_const} constant features)")



#METADATA SAMPLING DICTIONARIES


tier_metadata = defaultdict(lambda: defaultdict(list))
for r in meta_rows:
    mid = r["movie_id"]
    if mid not in tier_of_movie:
        continue
    for col in meta_cols:
        tier_metadata[tier_of_movie[mid]][col].append(r[col])

CATEGORICAL_SCENE = ["cut_count", "brightness", "motion_intensity", "audio_loudness",
                     "silence_ratio", "music_presence", "dialogue_density",
                     "face_screen_time_ratio", "lead_screen_time_ratio",
                     "budget_categorical"]



#POST-PROCESS SAMPLED FEATURES


def post_process_row(vec, feat_cols):
    """Force has_* masks → 1, round integer self-report features."""
    out = vec.copy().astype(float)
    for i, c in enumerate(feat_cols):
        if c in HAS_COLS:
            out[i] = 1.0
        elif c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_"):
            out[i] = round(np.clip(out[i], 1, 5))
        elif c in {"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"}:
            out[i] = round(np.clip(out[i], 1, 9))
        elif c.startswith("sr_emo_bc_"):
            out[i] = round(np.clip(out[i], -4, 4))
    return out



#GENERATE SYNTHETIC MOVIES


def sample_categorical(values):
    return random.choice(values)

n_per_tier = N_SYNTHETIC // 2
print(f"\nGenerating {N_SYNTHETIC} synthetic movies ({n_per_tier} per tier)...")

synthetic_metadata = []
synthetic_participants = []
synthetic_movie_features = []

syn_idx = 0
for tier in ["high", "lower"]:
    model = tier_models[tier]
    real_imdb = [float(v) for v in tier_metadata[tier]["imdb_rating"]]
    real_revenue = [int(v) for v in tier_metadata[tier]["worldwide_gross_revenue_usd_orignal_release"]]
    real_wom_log = [float(v) for v in tier_metadata[tier]["wom_multiplier_log"]]
    real_year = [int(v) for v in tier_metadata[tier]["release_year"]]
    real_duration = [int(v) for v in tier_metadata[tier]["clip_duration_s"]]

    imdb_min, imdb_max = min(real_imdb), max(real_imdb)
    rev_log_mean = statistics.mean([math.log10(v) for v in real_revenue])
    rev_log_std = statistics.stdev([math.log10(v) for v in real_revenue])
    wom_log_mean = statistics.mean(real_wom_log)
    wom_log_std = statistics.stdev(real_wom_log)

    for i in range(n_per_tier):
```

```
        syn_idx += 1
        syn_movie_id = f"SYN_M{syn_idx:03d}"
        syn_scene_id = f"SYN_S{syn_idx:03d}"

        # Sample 43 synthetic participants from the copula
        ppts = sample_copula(model, N_PARTICIPANTS)
        ppts = np.array([post_process_row(p, feat_cols) for p in ppts])

        syn_emotion = sample_categorical(tier_metadata[tier]["targeted_emotion"])
        syn_year = sample_categorical(real_year)
        syn_duration = sample_categorical(real_duration)
        syn_genre = sample_categorical(tier_metadata[tier]["genre_primary"])
        syn_genre2 = sample_categorical(tier_metadata[tier]["genre_secondary"])
        syn_country = sample_categorical(tier_metadata[tier]["country_of_origin"])
        syn_budget_cat = sample_categorical(tier_metadata[tier]["budget_categorical"])
        syn_imdb = round(random.uniform(imdb_min, imdb_max), 1)

        syn_rev = max(100_000, int(10 ** np.random.normal(rev_log_mean, rev_log_std)))
        syn_wom_log = np.random.normal(wom_log_mean, wom_log_std)
        syn_wom = 10 ** syn_wom_log
        syn_opening = max(10_000, int(syn_rev / syn_wom))

        cpi_year = CPI.get(syn_year, 200.0)
        inflate = CPI_BASE / cpi_year
        syn_rev_2025 = round(syn_rev * inflate, 2)
        syn_opn_2025 = round(syn_opening * inflate, 2)

        syn_categoricals = {col: sample_categorical(tier_metadata[tier][col])
                            for col in CATEGORICAL_SCENE}

        meta_row = {
            "scene_id": syn_scene_id,
            "movie_id": syn_movie_id,
            "movie_title": f"Synthetic_Movie_{syn_idx:03d}",
            "imdb_link": "",
            "targeted_emotion": syn_emotion,
            "clip_duration_s": syn_duration,
            "release_year": syn_year,
            "genre_primary": syn_genre,
            "genre_secondary": syn_genre2,
            "country_of_origin": syn_country,
            "imdb_rating": syn_imdb,
            "worldwide_gross_revenue_usd_orignal_release": syn_rev,
            "opening_weekend_usd": syn_opening,
            "revenue_usd_2025": syn_rev_2025,
            "opening_weekend_usd_2025": syn_opn_2025,
            "wom_multiplier": round(syn_wom, 4),
            "wom_multiplier_log": round(syn_wom_log, 4),
            "scene_notes": f"Synthetic — class-conditional copula sample from {tier} IMDb tier",
            "data_collection_notes": (
                f"Synthetic movie {syn_idx} of 40 (Gaussian copula sampling, "
                f"tier='{tier}', seed=42). Marginals match real distributions; "
                f"correlation structure preserved."
            ),
            "is_synthetic": 1,
        }
        meta_row.update(syn_categoricals)
        synthetic_metadata.append(meta_row)

        # Participant-level rows
        for p_i in range(N_PARTICIPANTS):
            syn_pid = SYNTHETIC_PARTICIPANT_ID_START + (syn_idx - 1) * N_PARTICIPANTS + p_i
            row = {"participant_id": syn_pid, "condition": syn_emotion, "movie_id": syn_movie_id}
            for j, c in enumerate(feat_cols):
                v = float(ppts[p_i, j])
                if abs(v) < 1e-6:
                    row[c] = 0
                else:
                    row[c] = round(v, 6)
            row["is_synthetic"] = 1
            synthetic_participants.append(row)

        # Movie-level aggregation
        m_row = {
            "movie_id": syn_movie_id,
            "condition": syn_emotion,
            "n_participants": N_PARTICIPANTS,
            "imdb_rating": syn_imdb,
```

```
            "wom_multiplier": round(syn_wom, 4),
            "wom_multiplier_log": round(syn_wom_log, 4),
            "is_synthetic": 1,
        }
        for j, c in enumerate(feat_cols):
            col_vals = ppts[:, j]
            m_row[f"{c}__mean"] = round(float(col_vals.mean()), 6)
            m_row[f"{c}__std"] = round(float(col_vals.std(ddof=1)), 6)
        synthetic_movie_features.append(m_row)



#WRITE OUTPUTS


syn_meta_cols = list(meta_cols) + ["is_synthetic"]
out_meta = DATA_DIR / "scene_movie_metadata_v6_synthetic.csv"
with open(out_meta, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_meta_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_metadata)
print(f"\nWrote {out_meta.name}: {len(synthetic_metadata)} rows × {len(syn_meta_cols)} cols")

syn_ppt_cols = list(ppt_cols) + ["is_synthetic"]
out_ppt = DATA_DIR / "participant_features_v6_synthetic.csv"
with open(out_ppt, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_ppt_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_participants)
print(f"Wrote {out_ppt.name}: {len(synthetic_participants)} rows × {len(syn_ppt_cols)} cols")

mov_cols, _ = load_csv(DATA_DIR / "movie_features_v6.csv")
syn_mov_cols = list(mov_cols)
if "is_synthetic" not in syn_mov_cols:
    syn_mov_cols.append("is_synthetic")
for c in ("imdb_rating", "wom_multiplier", "wom_multiplier_log"):
    if c not in syn_mov_cols:
        syn_mov_cols.append(c)

out_mov = DATA_DIR / "movie_features_v6_synthetic.csv"
with open(out_mov, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_mov_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_movie_features)
print(f"Wrote {out_mov.name}: {len(synthetic_movie_features)} rows × {len(syn_mov_cols)} cols")

# 8. VALIDATION SUMMARY

print("\n" + "=" * 60)
print("  VALIDATION — synthetic vs real distributions")
print("=" * 60)

real_imdb_all = list(movie_imdb.values())
syn_imdb_all = [m["imdb_rating"] for m in synthetic_metadata]
print(f"\nIMDb rating:")
print(f"  Real      (n={len(real_imdb_all)}): mean={statistics.mean(real_imdb_all):.2f}, "
      f"std={statistics.stdev(real_imdb_all):.2f}, "
      f"range={min(real_imdb_all):.1f}–{max(real_imdb_all):.1f}")
print(f"  Synthetic (n={len(syn_imdb_all)}): mean={statistics.mean(syn_imdb_all):.2f}, "
      f"std={statistics.stdev(syn_imdb_all):.2f}, "
      f"range={min(syn_imdb_all):.1f}–{max(syn_imdb_all):.1f}")

real_wom_log = [float(r["wom_multiplier_log"]) for r in meta_rows]
syn_wom_log = [m["wom_multiplier_log"] for m in synthetic_metadata]
print(f"\nWOM multiplier (log10):")
print(f"  Real      (n={len(real_wom_log)}): mean={statistics.mean(real_wom_log):.2f}, "
      f"std={statistics.stdev(real_wom_log):.2f}, "
      f"range={min(real_wom_log):.2f}–{max(real_wom_log):.2f}")
print(f"  Synthetic (n={len(syn_wom_log)}): mean={statistics.mean(syn_wom_log):.2f}, "
      f"std={statistics.stdev(syn_wom_log):.2f}, "
      f"range={min(syn_wom_log):.2f}–{max(syn_wom_log):.2f}")

print(f"\nDONE. Combined corpus (real + synthetic) = 50 movies.")
```

Simulate 10,000 movies (DL, n = 10,010 corpus)

ml_dataset/scripts/simulate_10k_movies_DL.py

```python
#!/usr/bin/env python3
"""
SIMULATION OF 10,000 SYNTHETIC MOVIES FOR DEEP LEARNING TRAINING

Simulate 10,000 synthetic movies using the same Gaussian copula
methodologyscaled up for deep-learning training.

Two-level hierarchical class-conditional sampling using
gaussian copulas.
The copula preserves BOTH:
  - Each feature's marginal distribution (by mapping through the empirical CDF)
  - The correlation structure between features (by sampling correlated normals)

Outputs 3 files in ml_dataset/data/model_ready/movie_success_v7/:
  - scene_movie_metadata_v7_synthetic.csv   (10,000 rows)
  - participant_features_v7_synthetic.csv   (10,000 × 43 = 430,000 rows)
  - movie_features_v7_synthetic.csv         (10,000 rows)

All synthetic rows include is_synthetic=1.
"""
from __future__ import annotations
import csv
import math
import random
import statistics
from pathlib import Path
from collections import Counter, defaultdict
import warnings

import numpy as np

warnings.filterwarnings("ignore")

PROJECT = Path(__file__).resolve().parent.parent.parent
INPUT_DIR  = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v6"
DATA_DIR   = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v7"
DATA_DIR.mkdir(parents=True, exist_ok=True)

RANDOM_SEED = 42
N_SYNTHETIC = 10000
N_PARTICIPANTS = 43
SYNTHETIC_PARTICIPANT_ID_START = 100000
TIER_THRESHOLD = 7.5
CPI_BASE = 322.0
CPI = {1978: 65.2, 1979: 72.6, 1983: 99.6, 1988: 118.3, 1993: 144.5,
       1996: 156.9, 1998: 163.0, 1999: 166.6, 2006: 201.6, 2012: 229.6,
       2025: 322.0}

random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)


def phi(z):
    """Standard normal CDF (vectorised over numpy array)."""
    z = np.asarray(z, dtype=float)
    return 0.5 * (1.0 + np.vectorize(math.erf)(z / math.sqrt(2.0)))


def phi_inv(p):
    """
    Inverse standard normal CDF using Beasley-Springer-Moro approximation.
    Vectorised over numpy array. Accuracy ~1e-9 in mid-range.
    """
    p = np.clip(np.asarray(p, dtype=float), 1e-12, 1 - 1e-12)

    a = [-39.6968302866538, 220.946098424521, -275.928510446969,
         138.357751867269, -30.6647980661472, 2.50662827745924]
    b = [-54.4760987982241, 161.585836858041, -155.698979859887,
         66.8013118877197, -13.2806815528857]
    c = [-0.00778489400243029, -0.322396458041136, -2.40075827716184,
         -2.54973253934373, 4.37466414146497, 2.93816398269878]
    d = [0.00778469570904146, 0.32246712907004, 2.445134137143, 3.75440866190742]

```

```
    p_low = 0.02425
    p_high = 1 - p_low
    out = np.empty_like(p)

    # Mid region
    mid = (p >= p_low) & (p <= p_high)
    if np.any(mid):
        q = p[mid] - 0.5
        r = q * q
        num = ((((a[0]*r + a[1])*r + a[2])*r + a[3])*r + a[4])*r + a[5]
        den = ((((b[0]*r + b[1])*r + b[2])*r + b[3])*r + b[4])*r + 1
        out[mid] = num * q / den

    # Lower tail
    low = p < p_low
    if np.any(low):
        q = np.sqrt(-2 * np.log(p[low]))
        num = ((((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
        den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
        out[low] = num / den

    # Upper tail (mirror of lower)
    high = p > p_high
    if np.any(high):
        q = np.sqrt(-2 * np.log(1 - p[high]))
        num = ((((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
        den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
        out[high] = -num / den

    return out

# COPULA UTILITIES

def empirical_ranks(values):
    """
    Return rank-based percentiles in (0, 1), avoiding boundaries.
    Uses (rank + 0.5) / n convention; tied values get the same rank.
    """
    n = len(values)
    if n == 0:
        return np.array([])
    # argsort-of-argsort gives 0-indexed rank
    ranks = np.argsort(np.argsort(values)).astype(float)
    return (ranks + 0.5) / n


def inverse_empirical(percentiles, sorted_values):
    """Linear-interpolating inverse ECDF: percentile [0,1] → value in original units."""
    n = len(sorted_values)
    if n == 0:
        return np.zeros_like(percentiles)
    if n == 1:
        return np.full_like(percentiles, sorted_values[0])
    idx = percentiles * (n - 1)
    idx = np.clip(idx, 0, n - 1)
    lo = np.floor(idx).astype(int)
    hi = np.minimum(lo + 1, n - 1)
    frac = idx - lo
    return sorted_values[lo] * (1.0 - frac) + sorted_values[hi] * frac


def fit_copula(X, regularize=0.05):
    """
    Fit a Gaussian copula on real data X (n × p).
    Returns model with: cholesky factor, sorted real values per feature,
                        and a flag indicating constant features.
    """
    n, p = X.shape

    sorted_per_feat = []
    constant_value = []                  # for features with zero variance
    is_constant = np.zeros(p, dtype=bool)

    Z = np.zeros((n, p))
    for j in range(p):
        col = X[:, j]
        valid_mask = ~np.isnan(col)
        valid = col[valid_mask]
```

```
        if len(valid) == 0:
            sorted_per_feat.append(np.array([0.0]))
            constant_value.append(0.0)
            is_constant[j] = True
            continue

        sorted_col = np.sort(valid)
        sorted_per_feat.append(sorted_col)

        # Constant feature
        if sorted_col[0] == sorted_col[-1]:
            is_constant[j] = True
            constant_value.append(sorted_col[0])
            Z[:, j] = 0.0
            continue
        constant_value.append(0.0)

        # Compute z-scores via empirical CDF → Φ⁻¹
        valid_ranks = empirical_ranks(valid)
        Z_valid = phi_inv(valid_ranks)

        # Place Z_valid back into the full column; impute NaN as 0 (median in z-space)
        z_col = np.zeros(n)
        z_col[valid_mask] = Z_valid
        z_col[~valid_mask] = 0.0
        Z[:, j] = z_col

    # Correlation matrix of Z
    # Skip constant columns when computing correlation
    var = Z.var(axis=0)
    nonconst = var > 1e-10
    Z_nc = Z[:, nonconst]
    C_nc = np.corrcoef(Z_nc, rowvar=False) if Z_nc.shape[1] > 1 else np.eye(1)
    C_nc = np.nan_to_num(C_nc, nan=0.0)

    # Reconstruct full correlation matrix: identity for constant columns
    C = np.eye(p)
    nc_indices = np.where(nonconst)[0]
    for i_idx, i in enumerate(nc_indices):
        for j_idx, j in enumerate(nc_indices):
            C[i, j] = C_nc[i_idx, j_idx]

    # Regularise: shrink toward identity for numerical stability
    C = (1 - regularize) * C + regularize * np.eye(p)

    # Cholesky decomposition (with extra regularisation if needed)
    for extra in [0.0, 0.05, 0.2, 0.5]:
        try:
            C_try = (1 - extra) * C + extra * np.eye(p)
            L = np.linalg.cholesky(C_try)
            break
        except np.linalg.LinAlgError:
            continue
    else:
        raise RuntimeError("Cholesky failed even with heavy regularisation.")

    return {
        "L": L, "p": p, "n": n,
        "sorted_per_feat": sorted_per_feat,
        "is_constant": is_constant,
        "constant_value": constant_value,
    }


def sample_copula(model, n_samples):
    """Draw n_samples × p synthetic feature vectors from the fitted copula."""
    L = model["L"]
    p = model["p"]
    sorted_per_feat = model["sorted_per_feat"]
    is_constant = model["is_constant"]
    constant_value = model["constant_value"]

    # Step 1: correlated standard normals
    noise = np.random.standard_normal((n_samples, p))
    Z_new = noise @ L.T

    # Step 2: → uniform via Φ
```

```
    U_new = phi(Z_new)

    # Step 3: → original space via inverse ECDF
    X_new = np.empty((n_samples, p))
    for j in range(p):
        if is_constant[j]:
            X_new[:, j] = constant_value[j]
        else:
            X_new[:, j] = inverse_empirical(U_new[:, j], sorted_per_feat[j])

    return X_new


def load_csv(path):
    with open(path) as f:
        reader = csv.DictReader(f)
        return reader.fieldnames, list(reader)

def to_float(s):
    if s is None or s == "" or s == "None":
        return float("nan")
    try:
        return float(s)
    except ValueError:
        return float("nan")

# 1. LOAD REAL DATA
print("Loading real data...")

meta_cols, meta_rows = load_csv(INPUT_DIR / "scene_movie_metadata_v6.csv")
print(f"  Metadata: {len(meta_rows)} movies × {len(meta_cols)} cols")

ppt_cols, ppt_rows = load_csv(INPUT_DIR / "participant_features_v6.csv")
print(f"  Participants: {len(ppt_rows)} rows × {len(ppt_cols)} cols")

movie_imdb = {r["movie_id"]: float(r["imdb_rating"]) for r in meta_rows
              if r.get("imdb_rating")}
tier_of_movie = {mid: ("high" if rating >= TIER_THRESHOLD else "lower")
                 for mid, rating in movie_imdb.items()}

movies_in_tier = defaultdict(list)
for mid, t in tier_of_movie.items():
    movies_in_tier[t].append(mid)

print("\nTier assignment:")
for t in ["high", "lower"]:
    mids = movies_in_tier[t]
    ratings = [movie_imdb[m] for m in mids]
    print(f"  {t:6s}: {len(mids)} movies, IMDb {min(ratings):.1f}–{max(ratings):.1f}")

# 2. PREPARE PARTICIPANT-LEVEL FEATURES PER TIER

ID_COLS = {"participant_id", "condition", "movie_id"}
HAS_COLS = {c for c in ppt_cols if c.startswith("has_")}
SR_INTEGER_COLS = {c for c in ppt_cols
                   if c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_")}
SR_INTEGER_COLS.update({"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"})

feat_cols = [c for c in ppt_cols if c not in ID_COLS]
n_feat = len(feat_cols)

tier_data = defaultdict(list)
for r in ppt_rows:
    mid = r["movie_id"]
    if mid not in tier_of_movie:
        continue
    vec = np.array([to_float(r[c]) for c in feat_cols])
    tier_data[tier_of_movie[mid]].append(vec)

print(f"\nFeature matrix sizes:")
for t, vecs in tier_data.items():
    arr = np.vstack(vecs)
    print(f"  Tier '{t}': {arr.shape[0]} × {arr.shape[1]}, NaN rate {np.isnan(arr).mean():.1%}")

# 3. FIT GAUSSIAN COPULAS

print("\nFitting Gaussian copulas (preserves marginals + correlations)...")
tier_models = {}
```

```python
for t, vecs in tier_data.items():
    X = np.vstack(vecs)
    model = fit_copula(X, regularize=0.05)
    tier_models[t] = model
    n_const = int(model["is_constant"].sum())
    print(f"  Tier '{t}': fitted on {model['n']} rows × {model['p']} features  "
          f"({n_const} constant features)")


# 4. METADATA SAMPLING DICTIONARIES

tier_metadata = defaultdict(lambda: defaultdict(list))
for r in meta_rows:
    mid = r["movie_id"]
    if mid not in tier_of_movie:
        continue
    for col in meta_cols:
        tier_metadata[tier_of_movie[mid]][col].append(r[col])

CATEGORICAL_SCENE = ["cut_count", "brightness", "motion_intensity", "audio_loudness",
                     "silence_ratio", "music_presence", "dialogue_density",
                     "face_screen_time_ratio", "lead_screen_time_ratio",
                     "budget_categorical"]


# 5. POST-PROCESS SAMPLED FEATURES

def post_process_row(vec, feat_cols):
    """Force has_* masks → 1, round integer self-report features."""
    out = vec.copy().astype(float)
    for i, c in enumerate(feat_cols):
        if c in HAS_COLS:
            out[i] = 1.0
        elif c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_"):
            out[i] = round(np.clip(out[i], 1, 5))
        elif c in {"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"}:
            out[i] = round(np.clip(out[i], 1, 9))
        elif c.startswith("sr_emo_bc_"):
            out[i] = round(np.clip(out[i], -4, 4))
    return out

# 6. GENERATE SYNTHETIC MOVIES

def sample_categorical(values):
    return random.choice(values)

n_per_tier = N_SYNTHETIC // 2
print(f"\nGenerating {N_SYNTHETIC} synthetic movies ({n_per_tier} per tier)...")

synthetic_metadata = []
synthetic_participants = []
synthetic_movie_features = []

syn_idx = 0
for tier in ["high", "lower"]:
    model = tier_models[tier]
    real_imdb = [float(v) for v in tier_metadata[tier]["imdb_rating"]]
    real_revenue = [int(v) for v in tier_metadata[tier]["worldwide_gross_revenue_usd_orignal_release"]]
    real_wom_log = [float(v) for v in tier_metadata[tier]["wom_multiplier_log"]]
    real_year = [int(v) for v in tier_metadata[tier]["release_year"]]
    real_duration = [int(v) for v in tier_metadata[tier]["clip_duration_s"]]

    imdb_min, imdb_max = min(real_imdb), max(real_imdb)
    rev_log_mean = statistics.mean([math.log10(v) for v in real_revenue])
    rev_log_std = statistics.stdev([math.log10(v) for v in real_revenue])
    wom_log_mean = statistics.mean(real_wom_log)
    wom_log_std = statistics.stdev(real_wom_log)

    for i in range(n_per_tier):
        syn_idx += 1
        syn_movie_id = f"SYN_M{syn_idx:03d}"
        syn_scene_id = f"SYN_S{syn_idx:03d}"

        # Sample 43 synthetic participants from the copula
        ppts = sample_copula(model, N_PARTICIPANTS)
        ppts = np.array([post_process_row(p, feat_cols) for p in ppts])

        syn_emotion = sample_categorical(tier_metadata[tier]["targeted_emotion"])
```

```
        syn_year = sample_categorical(real_year)
        syn_duration = sample_categorical(real_duration)
        syn_genre = sample_categorical(tier_metadata[tier]["genre_primary"])
        syn_genre2 = sample_categorical(tier_metadata[tier]["genre_secondary"])
        syn_country = sample_categorical(tier_metadata[tier]["country_of_origin"])
        syn_budget_cat = sample_categorical(tier_metadata[tier]["budget_categorical"])
        syn_imdb = round(random.uniform(imdb_min, imdb_max), 1)

        syn_rev = max(100_000, int(10 ** np.random.normal(rev_log_mean, rev_log_std)))
        syn_wom_log = np.random.normal(wom_log_mean, wom_log_std)
        syn_wom = 10 ** syn_wom_log
        syn_opening = max(10_000, int(syn_rev / syn_wom))

        cpi_year = CPI.get(syn_year, 200.0)
        inflate = CPI_BASE / cpi_year
        syn_rev_2025 = round(syn_rev * inflate, 2)
        syn_opn_2025 = round(syn_opening * inflate, 2)

        syn_categoricals = {col: sample_categorical(tier_metadata[tier][col])
                            for col in CATEGORICAL_SCENE}

        meta_row = {
            "scene_id": syn_scene_id,
            "movie_id": syn_movie_id,
            "movie_title": f"Synthetic_Movie_{syn_idx:03d}",
            "imdb_link": "",
            "targeted_emotion": syn_emotion,
            "clip_duration_s": syn_duration,
            "release_year": syn_year,
            "genre_primary": syn_genre,
            "genre_secondary": syn_genre2,
            "country_of_origin": syn_country,
            "imdb_rating": syn_imdb,
            "worldwide_gross_revenue_usd_orignal_release": syn_rev,
            "opening_weekend_usd": syn_opening,
            "revenue_usd_2025": syn_rev_2025,
            "opening_weekend_usd_2025": syn_opn_2025,
            "wom_multiplier": round(syn_wom, 4),
            "wom_multiplier_log": round(syn_wom_log, 4),
            "scene_notes": f"Synthetic — class-conditional copula sample from {tier} IMDb tier",
            "data_collection_notes": (
                f"Synthetic movie {syn_idx} of 40 (Gaussian copula sampling, "
                f"tier='{tier}', seed=42). Marginals match real distributions; "
                f"correlation structure preserved."
            ),
            "is_synthetic": 1,
        }
        meta_row.update(syn_categoricals)
        synthetic_metadata.append(meta_row)

        # Participant-level rows
        for p_i in range(N_PARTICIPANTS):
            syn_pid = SYNTHETIC_PARTICIPANT_ID_START + (syn_idx - 1) * N_PARTICIPANTS + p_i
            row = {"participant_id": syn_pid, "condition": syn_emotion, "movie_id": syn_movie_id}
            for j, c in enumerate(feat_cols):
                v = float(ppts[p_i, j])
                if abs(v) < 1e-6:
                    row[c] = 0
                else:
                    row[c] = round(v, 6)
            row["is_synthetic"] = 1
            synthetic_participants.append(row)

        # Movie-level aggregation
        m_row = {
            "movie_id": syn_movie_id,
            "condition": syn_emotion,
            "n_participants": N_PARTICIPANTS,
            "imdb_rating": syn_imdb,
            "wom_multiplier": round(syn_wom, 4),
            "wom_multiplier_log": round(syn_wom_log, 4),
            "is_synthetic": 1,
        }
        for j, c in enumerate(feat_cols):
            col_vals = ppts[:, j]
            m_row[f"{c}__mean"] = round(float(col_vals.mean()), 6)
            m_row[f"{c}__std"] = round(float(col_vals.std(ddof=1)), 6)
        synthetic_movie_features.append(m_row)
```

```python

# 7. WRITE OUTPUTS

syn_meta_cols = list(meta_cols) + ["is_synthetic"]
out_meta = DATA_DIR / "scene_movie_metadata_v7_synthetic.csv"
with open(out_meta, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_meta_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_metadata)
print(f"\nWrote {out_meta.name}: {len(synthetic_metadata)} rows × {len(syn_meta_cols)} cols")

syn_ppt_cols = list(ppt_cols) + ["is_synthetic"]
out_ppt = DATA_DIR / "participant_features_v7_synthetic.csv"
with open(out_ppt, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_ppt_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_participants)
print(f"Wrote {out_ppt.name}: {len(synthetic_participants)} rows × {len(syn_ppt_cols)} cols")

mov_cols, _ = load_csv(INPUT_DIR / "movie_features_v6.csv")
syn_mov_cols = list(mov_cols)
if "is_synthetic" not in syn_mov_cols:
    syn_mov_cols.append("is_synthetic")
for c in ("imdb_rating", "wom_multiplier", "wom_multiplier_log"):
    if c not in syn_mov_cols:
        syn_mov_cols.append(c)

out_mov = DATA_DIR / "movie_features_v7_synthetic.csv"
with open(out_mov, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=syn_mov_cols, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(synthetic_movie_features)
print(f"Wrote {out_mov.name}: {len(synthetic_movie_features)} rows × {len(syn_mov_cols)} cols")

# 8. VALIDATION SUMMARY

print("\n" + "=" * 60)
print("  VALIDATION — synthetic vs real distributions")
print("=" * 60)

real_imdb_all = list(movie_imdb.values())
syn_imdb_all = [m["imdb_rating"] for m in synthetic_metadata]
print(f"\nIMDb rating:")
print(f"  Real      (n={len(real_imdb_all)}): mean={statistics.mean(real_imdb_all):.2f}, "
      f"std={statistics.stdev(real_imdb_all):.2f}, "
      f"range={min(real_imdb_all):.1f}–{max(real_imdb_all):.1f}")
print(f"  Synthetic (n={len(syn_imdb_all)}): mean={statistics.mean(syn_imdb_all):.2f}, "
      f"std={statistics.stdev(syn_imdb_all):.2f}, "
      f"range={min(syn_imdb_all):.1f}–{max(syn_imdb_all):.1f}")

real_wom_log = [float(r["wom_multiplier_log"]) for r in meta_rows]
syn_wom_log = [m["wom_multiplier_log"] for m in synthetic_metadata]
print(f"\nWOM multiplier (log10):")
print(f"  Real      (n={len(real_wom_log)}): mean={statistics.mean(real_wom_log):.2f}, "
      f"std={statistics.stdev(real_wom_log):.2f}, "
      f"range={min(real_wom_log):.2f}–{max(real_wom_log):.2f}")
print(f"  Synthetic (n={len(syn_wom_log)}): mean={statistics.mean(syn_wom_log):.2f}, "
      f"std={statistics.stdev(syn_wom_log):.2f}, "
      f"range={min(syn_wom_log):.2f}–{max(syn_wom_log):.2f}")

print(f"\nDONE. Combined corpus (real + synthetic) = 50 movies.")
```